CTO용 Arc Reactor 활용 가이드

작성일: 2026-04-06
대상: CTO / C-level 의사결정권자

0) 자동 분석 블록 (2분 간격 동기화)

이 섹션은 2분마다 cron으로 자동 재생성됩니다.

- 마지막 동기화: 2026-04-06 08:24:29 +0900
- 분석 범위: /Users/jinan/ai 내 아슬란 생태계 프로젝트 8개 (arc-reactor, aslan-iam, swagger-mcp-server, atlassian-mcp-server, clipping-mcp-server, arc-reactor-admin, arc-reactor-web, Aslan-Verse-Web)
- 생성 방식: README/ECOSYSTEM.md 기반 정합성 스냅샷 + git 메타데이터

0) 작성 규칙(문체/근거/의사결정)

- 톤: CTO 의사결정 보고용 한국어 존댓말(입니다/됩니다) 사용
- 근거 우선: 각 주장 끝에 구현 파일 경로(가능 시 라인) 또는 git 상태 근거를 붙입니다.
- 요약 우선: 비즈니스 영향 → 운영 리스크 → 실행 결정의 3단 구조를 유지합니다.
- 미확인 항목 표기: "미확인(운영 연동/실행 검증 필요)"로 분리합니다.
- 문체: 과장 표현 없이 확인된 근거 기반으로만 기술합니다.

1) CTO 의사결정 요약

- 판단 프레임: arc-reactor 중심으로 IAM 인증, MCP 동적 연동, 채널 확장, 관리자 제어를 묶는 중앙 제어형 아키텍처입니다.
- 현재 판단 근거: `POST /api/mcp/servers` 동적 등록 체계, Arc Reactor MCP allowlist, aslan-iam 공개키 기반 인증, arc-reactor-admin 운영 콘솔, 웹/슬랙 채널 경로가 문서 및 코드에 존재합니다.
- 의사결정 포인트: 우선 도입 범위를 "운영 안정화가 검증된 핵심 경로"로 좁혀 단계적으로 롤아웃하는 쪽이 리스크 최소화가 유리합니다.

1-1) 아슬란 통합 철학(CTO 판단 포인트)

- 아슬란의 목표는 흩어진 업무를 하나의 운영 지점으로 정렬하는 것입니다. 인증·커뮤니티·클리핑·향후 확장 서비스가 Reactor를 통해 같은 정책 언어로 연결되면, 신규 서비스 오픈 속도는 올라가고 통제면은 유지됩니다.
- 근거:
  - 인증은 `aslan-iam`이 중앙에서 처리하고, Reactor는 인증 토큰/권한 체계를 전파해 사용/도구 레이어에서 재검증합니다.
  - 기능 연동은 MCP 등록 API(`POST /api/mcp/servers`)와 allowlist 기반 제어로 표준화되어 있어 신규 외부 서비스의 편입 절차가 단순합니다.
  - 운영/보안은 `arc-reactor-admin`와 Guard/Hook 정책으로 같은 프레임에서 감시할 수 있어 서비스별 이기종 운영의 위험을 줄입니다.
  - Reactor Work/Workflow 빌더 전략을 적용하면, 승인/감사/비용 통제 지점이 내장된 업무 흐름을 팀 단위로 템플릿화해 반복 구축 비용을 줄일 수 있습니다.
- 제안 우선순위:
  - 1단계: 핵심(인증·웹·MCP·운영 UI) 우선 통합
  - 2단계: 커뮤니티/클리핑/협업형 MCP를 동일 정책 템플릿으로 연결
  - 3단계: 신규 서비스별 팀형 리액터(페르소나) 병렬 시범 운영 후 단계적 확장
- 경영 판단: "모든 신규 서비스는 Reactor 기준선 + 독립 비즈니스 기능"의 구조로 설계하면, 기술부채보다 제어력(컴플라이언스·보안·운영 가시성)을 먼저 확보할 수 있습니다.

2) 프로젝트 상태 스냅샷

프로젝트	경로	역할	상태	README 제목	브랜치	커밋	워크트리
arc-reactor	/Users/jinan/ai/arc-reactor	핵심 런타임	OK	Arc Reactor	main	1773c962	dirty

프로젝트	경로	역할	상태	README 제목	브랜치	커밋	워크트리
aslan-iam	/Users/jinan/ai/aslan-iam	중앙 인증(IAM)	OK	aslan-iam	main	ee88eb3	dirty
swagger-mcp-server	/Users/jinan/ai/swagger-mcp-server	API 도구 MCP	OK	Swagger MCP Server	main	8964374	clean
atlassian-mcp-server	/Users/jinan/ai/atlassian-mcp-server	Jira/Confluence/Bitbucket MCP	OK	atlassian-mcp-server	main	054a127	clean
clipping-mcp-server	/Users/jinan/ai/clipping-mcp-server	클리핑·요약 MCP	OK	clipping-mcp-server	main	23abc52	clean
arc-reactor-admin	/Users/jinan/ai/arc-reactor-admin	운영/모니터링 UI	OK	arc-reactor-admin	main	ca42355	clean
arc-reactor/arc-web	/Users/jinan/ai/arc-reactor/arc-web	채팅/REST 모듈(내부)	OK	-	main	1773c962	dirty
arc-reactor/arc-slack	/Users/jinan/ai/arc-reactor/arc-slack	슬랙 채널 모듈(내부)	OK	-	main	1773c962	dirty
arc-reactor-web	/Users/jinan/ai/arc-reactor-web	사용자 UI	OK	Arc Reactor Web	main	e0d84a8	clean
Aslan-Verse-Web	/Users/jinan/ai/Aslan-Verse-Web	실험형 멀티 페르소나 조직 플랫폼	OK	-	main	c20bbdb	clean

3) CTO 운영 관점 정합성 점검

- 핵심 연동 점검: curl -X POST http://localhost:18081/api/mcp/servers
- 보안/정책 게이트 점검: | ARC_REACTOR_MCP_ALLOWED_SERVER_NAMES | Comma-separated MCP server allowlist (e.g. atlassian,swagger) |
- API/토큰 운영 포인트: POST /api/mcp/servers, PUT /api/mcp/servers/{name}로 등록하고 갱신합니다.
- 인증 연동 점검: JWT 토큰 발급의 유일한 권한을 가지며, 타 서비스는 공개키로 로컬 검증만 수행합니다.
- 실험 플랫폼 연계: 가상 휴넷(Virtual Human Network) — AI 페르소나들이 실제 회사처럼 역할을 맡아 협업하는 플랫폼. 사용자는 AI 팀원과 함께 실무를 진행하고, 조직 구조와 업무 현황을 시각화하여 모니터링한다.

4) 핵심 제어 포인트(근거 기반)

- 가드/보안 제어 근거: /Users/jinan/ai/arc-reactor/arc-core/src/main/kotlin/com/arc/reactor/tool/idempotency/InMemoryToolIdempotencyGuard.kt:21:class InMemoryToolIdempotencyGuard(
- 운영 정책 제어 근거: /Users/jinan/ai/arc-reactor/arc-core/src/test/kotlin/com/arc/reactor/approval/ToolApprovalPolicyTest.kt:8: * ToolApprovalPolicy에 대한 테스트.
- 리스크: 외부 MCP/챗봇 의존도 증가 시 장애 영향면이 커질 수 있어 실패 격리(Guard/Hook/Timeout)와 운영 알림을 선행해야 합니다.
- 기대효과: 중앙 인증 분리 + 표준 MCP 등록 흐름으로 팀별 Agent 실험(Aslan-Verse-Web)과 비즈니스 채널을 분리 배포할 수 있습니다.
- 의사결정 포인트(3개):
  - 1차 8주 내 도입 범위를 MCP/관리자 UI와 웹 채널에 한정해 안정성 우선 구축할지 결정
  - 가드 정책(도구 승인/속도 제한/오류 격리)은 fail-close를 고정할지, 최소 기능 집합부터 단계 적용할지 결정
  - 운영 KPI(일일 요청 수, 도구 실패율, 정책 우회율) 기준으로 점진 확장 여부를 결정

5) 2분 주기 변경 감시(요약)

- arc-reactor (핵심 런타임) : branch main, 최근 2분 변경 0건
- aslan-iam (중앙 인증(IAM)) : branch main, 최근 2분 변경 0건

- swagger-mcp-server (API MCP) : branch main, 최근 2분 변경 0건
- atlassian-mcp-server (Atlassian MCP) : branch main, 최근 2분 변경 0건
- clipping-mcp-server (Clipping MCP) : branch main, 최근 2분 변경 0건
- arc-reactor-admin (운영 UI) : branch main, 최근 2분 변경 0건
- arc-reactor-web (웹 채널) : branch main, 최근 2분 변경 0건
- arc-reactor/arc-web (채팅/REST 모듈) : branch main, 최근 2분 변경 0건
- arc-reactor/arc-slack (슬랙 모듈) : branch main, 최근 2분 변경 0건
- Aslan-Verse-Web (실험형 조직 플랫폼) : branch main, 최근 2분 변경 0건

0) 핵심 메시지

Arc Reactor는 "챗봇 하나"가 아니라,
AI를 안전하고 제어된 방식으로 조직 전반에서 운영하는 런타임 + 거버넌스 플랫폼입니다.

우리의 목표는 단순히 질의 응답을 빠르게 하는 것이 아니라,

- 위험한 행위를 자동으로 차단하고,
- 운영자 승인 흐름을 내장하고,
- 감사 가능한 로그와 비용·성능 통제를 제공하며,
- 웹/Slack/MCP/실험 플랫폼을 하나의 정책 레이어에서 통합하는 것입니다.

1) CTO가 보는 핵심 가치

1.1 규제 대응과 책임성

- 인증·권한 경로가 중앙화됨(`aslan-iam` 연동)
- 도구 실행 전 `Guard`에서 위험 제어(실패 시 즉시 차단)
- 위험 동작은 `HITL`로 사람 승인 전환 가능
- 출력 가드/PII 마스킹으로 대외 유출 리스크 감소
- 감사 로그로 사고 시 포렌식 추적이 가능

1.2 비용 통제

- LLM 호출 횟수·길이·재시도·도구 호출을 정책으로 제어
- 스케줄러로 반복 업무 자동화 비용을 예측 가능하게 운영
- 실패/재시도 패턴을 모니터링해 조기 차단
- 대형 업무군을 하나의 운영 정책으로 합치면 비용 정산 라인 단순화

1.3 서비스 안정성

- Hook가 실패해도 운영은 멈추지 않는 fail-open, Guard는 fail-close로 안전성 유지
- ReAct 루프 종료 조건이 명시되어 있어서 무한반복 위험 감소
- 스케줄링, Slack 이벤트, 웹 채널이 같은 엔진 정책을 공유해 운영 편차 감소

1.4 확장성

- MCP를 런타임에서 동적으로 등록 가능(새 내부 시스템 연동이 빠름)
- 팀별/서비스별로 persona와 tool 정책을 다르게 구성해 "한 플랫폼, 여러 조직" 운영
- `Aslan-Verse-Web` 실험에서 팀별 멀티 리액터/페르소나를 곧바로 검증 가능

2) 왜 지금 기업용으로 유리한가

2.1 이미 연결되어 있는 자산을 곧바로 활용

- 중앙 인증: `aslan-iam` 재사용으로 별도 인증 스택 구축 비용 절감

- 도구 생태계: `swagger-mcp-server`, `atlassian-mcp-server`, `clipping-mcp-server` 즉시 연결
- 관리자 레이어: `arc-reactor-admin`에서 정책·승인·실시간 상태를 한 화면에서 관리
- 채널 커버리지: 웹과 Slack을 동시에 지원해 별도 통합 비용 감소

2.2 조직 학습 비용 감소

- Arc Reactor의 `persona`와 `promptTemplate`가 정책 레이어와 결합되어, 새로운 팀/부서 온보딩이 빠름
- 채널별 동작 일관성으로 운영팀이 동일한 관제 방식으로 관리 가능
- 문서·보고 형태로 결과물을 바로 공유 가능(기획서, 감사 보고, 임원 브리핑)

3) 와우 포인트 (임원 보고용 하이라이트)

와우 포인트 1. "AI를 허가된 방식으로만 쓰는 조직"

일반 LLM 사용은 보통 "누가 무엇을 실행했는지" 추적이 어렵습니다.
Arc Reactor는 실행 전 Guard + 실행 후 Audit로 "제어된 AI"를 제공합니다.

와우 포인트 2. "한 번의 정책으로 채널 통합"

웹채팅/Slack/실험형 `Aslan-Verse-Web`이 따로따로 운영되지 않고, 동일한 정책 집합을 공유합니다.

와우 포인트 3. "실험 조직까지 본격 엔터프라이즈화"

Aslan-Verse-Web에서 가상 팀을 생성해 팀별 페르소나·행동 양식을 실험하고, 성능/품질/비용을 동일 기준으로 비교 가능.

와우 포인트 4. "2분 주기 정책 반영"

문서·운영 아티팩트 자동 생성(본 작업), 그리고 스케줄러/크론 기반 정기 실행으로 운영 신뢰도와 최신성이 유지됩니다.

4) 핵심 사용 시나리오(CTO 관점)

A) 보안 강화형 AI 운영 도입

- `Guard`를 보수적 값으로 시작해 정책을 단계적으로 확장
- 승인 대상 도구를 제한하고, 위험 Tool은 HITL로 이관
- 출력 결과 검수를 위한 outputGuard + 관리자 리포트 채널 운영

B) 내부 업무 자동화 확장

- MCP로 사내 API/Jira/Confluence/클리핑 체인을 연결
- 반복 리포트/요약/스캔 작업은 스케줄러 기반으로 자동화
- Slack에서 요청하면 승인 가능한 형태로 즉시 실행

C) 멀티 팀 실험 플랫폼

- 각 팀별 리액터 인스턴스를 persona 기반으로 분기
- 동일 과제에 대한 응답 스타일, 리스크 대응, 실행속도를 비교
- 성능 지표를 기반으로 최적 전략 선별

5) 의사결정자가 확인해야 할 KPI

- 규제/보안:
  - 승인되지 않은 tool 실행 건수 0
  - PII 마스킹/출력 가드 위반 건수

- 운영:
  - Guard 차단율(너무 과도하면 정책 완화, 너무 낮으면 완화 필요)
  - ReAct 반복/시간 초과율
- 비용:
  - 세션/도구 호출당 토큰 소모 추이
  - 스케줄러 작업의 단가 대비 산출물 비율
- 사용자 체감:
- 평균 응답 지연, 에러율, 재요청률

6) 90일 도입 로드맵 제안

1~30일: 안전 베이스 구축

- aslan-iam 연동, 인증·권한 연동, 기본 Guard 정책
- 웹/Slack 채널 최소 기능 오픈
- `arc-reactor-admin` 대시보드 기본 상태 점검 체계 구성

31~60일: 생산성 레이어

- swagger/atlassian/clipping MCP 등록 및 운영 승인 규칙 정리
- 핵심 업무에만 제한된 도구 허용
- 자동 리포트/요약 파이프라인 도입

61~90일: 임원 보고/실험

- Aslan-Verse-Web 연동 실험을 통해 팀별 퍼포먼스 비교
- 비용·품질·안전 지표 기반 정책 고도화
- CTO 브리핑용 KPI 리포트 정기 발행 구조 완성

7) 즉시 실행 요청(권장)

1. 본 문서 기준으로 정책 스펙(보안/출력/승인) 사전 승인
2. 자동화 문서 파이프라인(cron 2분)에 대한 운영 권한 위임
3. Aslan-Verse-Web PoC 팀 2~3개 선정(예: 기획/개발/영업 지원형)
4. 4주 단위 리뷰로 tool 사용 승인 정책을 조정

8) 결론

Arc Reactor는 “개발자가 만든 챗봇”이 아니라,
법무·보안·재무·조직 운영이 공존할 수 있는 AI 운영 플랫폼입니다.
이 구조를 쓰면 기술 도입 속도는 유지하면서도 통제는 강화할 수 있고,
실험적 조직 운영(Aslan-Verse-Web)까지 한 번의 거버넌스로 확장 가능합니다.